

Food Fiesta

Premium Dining Management Application

Generated on: 17 juillet 2026 | Spring Boot 3.4.2 | Java 21

TABLE OF CONTENTS

- 1. Purpose & Overview
- 2. How to Start the Application
- 3. Application Architecture
- 4. Architecture Diagram
- 5. CI/CD & Deployment
- 6. Project Health & Technical Debt Tracking
- 7. Known Gotchas & Contribution Guide

1. Purpose & Overview

Food Fiesta is a full-stack dining management web application built with Java 21 and Spring Boot 3.4.2. It allows a restaurant to manage its product catalog (dishes), customer accounts, and food orders through a web interface, while providing customers with a self-service ordering experience.

User Roles

ROLE	WHAT THEY CAN DO
Admin	Full back-office access: manage customers, products, and orders; add/delete/update admins; view all orders.

Customer (User)	Browse the menu, register an account, log in (form or Google OAuth2), search for dishes, place orders, and view order history.
------------------------	--

Core Features

- Premium Thymeleaf UI with custom CSS per page
- Role-based dual authentication (admin vs. user flows)
- Product catalog with CRUD management
- Order placement with total calculation via `Logic.countTotal()`
- Order history per user
- Optional Google OAuth2 login
- Swagger/OpenAPI documentation at `/swagger-ui/index.html`
- H2 in-memory DB for development; PostgreSQL-ready for production

2. How to Start the Application

Prerequisites

TOOL	VERSION	NOTES
JDK	21	Required — Maven Wrapper is included, no standalone Maven needed
Docker	Any recent	Optional — only for PostgreSQL/Docker setup
PostgreSQL	Any recent	Optional — only if replacing H2

Quick Start (H2 — Recommended for local dev)

```
# Windows PowerShell
.\mvnw.cmd spring-boot:run

# If JAVA_HOME is not set
$env:JAVA_HOME="C:\Program Files\Java\jdk-21.0.11"
.\mvnw.cmd spring-boot:run

# Linux / macOS
./mvnw spring-boot:run
```

Local URLs

URL	DESCRIPTION
<code>http://localhost:8080/</code>	Application homepage
<code>http://localhost:8080/swagger-ui/index.html</code>	Swagger / OpenAPI documentation
<code>http://localhost:8080/h2-console</code>	H2 database console (dev only)

H2 Console Connection

```
JDBC URL: jdbc:h2:mem:foodfiesta
User Name: sa
Password: (leave empty)
```

Note: The H2 database is in-memory. All data resets when the application stops. On the next startup, `DataLoader` re-seeds 5 products and 1 default admin automatically.

Default Seeded Credentials

ROLE	EMAIL	PASSWORD
Admin	<code>admin@foodfiesta.com</code>	<code>admin123</code>

Security: Passwords are stored in *plain text*. `BCryptPasswordEncoder` is declared as a bean but is not wired to any save/validation path. Do **not** change the default password format without updating both `AdminServices.validateAdminCredentials()` and `UserServices.validateLoginCredentials()`.

Run Tests

```
.\mvnw.cmd test
```

Package (skip tests)

```
.\mvnw.cmd clean package -DskipTests
```

Environment Variables / Config to Set

The file `src/main/resources/application.properties` is the single config file. For local H2 development, no changes are needed. For Google OAuth2, replace the two placeholders:

```
spring.security.oauth2.client.registration.google.client-id=YOUR_CLIENT_ID
spring.security.oauth2.client.registration.google.client-secret=YOUR_CLIENT_SECRET
```

Leaving them as-is disables Google login but does *not* break the form-based login.

3. Application Architecture

Layer Overview

LAYER	PACKAGE	ROLE
Controllers	<code>com.example.demo.controllers</code>	Handle HTTP requests, bind session, return Thymeleaf view names
Services	<code>com.example.demo.services</code>	Business logic — annotated <code>@Component</code> (not <code>@Service</code>)
Repositories	<code>com.example.demo.repositories</code>	Spring Data interfaces extending <code>CrudRepository</code>
Entities	<code>com.example.demo.entities</code>	JPA entities mapped to H2/PostgreSQL tables
Config	<code>com.example.demo.config</code>	Security configuration, seed data loader, OpenAPI config
Count	<code>com.example.demo.count</code>	Utility class <code>Logic</code> for order total calculation
loginCredentials	<code>com.example.demo.loginCredentials</code>	DTOs for form binding (<code>AdminLogin</code> , <code>UserLogin</code>)

Authentication Mechanism

The project uses a **hybrid authentication** approach: Spring Security handles route access control and OAuth2 (Google), while custom session-based auth handles form login.

- **Admin login** — POST `/adminLogin` → `AdminController.getAllData()` calls `AdminServices.validateAdminCredentials(email, password)` (plain-text comparison). On success: stores the email string in session as `"loggedInAdmin"`.

- **User login** — POST `/userLogin` → `AdminController.userLogin()` calls `UserServices.validateLoginCredentials(email, password)` (plain-text). On success: stores the full `User` JPA entity in session as `"loggedInUser"`.
- **Google OAuth2** — redirect to `/oauth2/success` → `OAuth2Controller` finds or creates a user, sets `upassword="OAUTH_USER"` and `unumber=0` as placeholders, then puts the `User` object in session.
- **Logout** — GET `/logout` → session invalidated, redirects to `/home`.

Data Model

ENTITY	DB TABLE	KEY FIELDS	NOTES
User	customer_info	u_id, uname, uemail, upassword, unumber	Field <code>u_id</code> uses underscore. Getter: <code>getU_id()</code> . Has <code>@OneToMany</code> to Orders.
Admin	admin	adminId, adminName, adminEmail, adminPassword, adminNumber	Plain-text password. Bug in <code>AdminServices.update()</code> : does not set ID on the admin object before saving.
Product	product_table	pid, pname, pprice, pdescription	Seeded with 5 dishes on startup.
Orders	order_info	oId, oName, oPrice, oQuantity, orderDate, totalAmmout	Intentional typo: <code>totalAmmout</code> (double-m). Never "fix" it — it maps to the DB column. Uses <code>GenerationType.AUTO</code> .

Request Lifecycle

Browser → **Spring Security** (route guard) → **Controller** (session check, model binding) → **Service** (`@Component`, business logic) → **Repository** (`CrudRepository`) → **H2 / PostgreSQL** → Controller returns Thymeleaf view name → **Browser** renders HTML.

Non-Obvious Patterns

- Services are annotated `@Component`, not `@Service` — this is intentional, do not change it.
- `findAll()` on all repositories returns `Iterable<T>` (`CrudRepository`). Always cast to `List<T>` explicitly.
- All mutation routes (update/delete) use `@GetMapping` — this is the established pattern, do not change to PUT/DELETE.
- Order total must always use `Logic.countTotal(price, quantity)` — never inline the multiplication.
- `OrderController.java` is an empty stub — order logic lives in `AdminController`.
- `AdminServices.update()` has a bug: it iterates admins, finds the matching ID, but calls `save(admin)` without setting `admin.setAdminId(id)` first. The update saves a new row instead of updating.

- Both `AdminServices.getAdmin()` and `ProductServices.getProduct()` call `optional.get()` without a guard — throws `NoSuchElementException` if the ID does not exist.

4. Architecture Diagram

🌐 CLIENT / BROWSER

Thymeleaf SSR (server-side rendering)

HTML Forms (POST/GET)

CSS + JavaScript (static assets)

↓
HTTP Request

🔒 SPRING SECURITY + CUSTOM SESSION AUTH — ⚠️ PLAIN-TEXT PASSWORDS / BCrypt NOT APPLIED

SecurityConfig (CSRF disabled)

Spring OAuth2 Client (Google)

BCryptPasswordEncoder ⚠️ declared but unused

Session guard (loggedInAdmin / loggedInUser)

↓
Authorized requests routed to controllers

🎮 CONTROLLERS

HomeController

AdminController (auth + dashboard + orders)

UserController

ProductController

OAuth2Controller

OrderController ⚠️ empty stub

↓
Service calls

⚙️ SERVICES — ALL ANNOTATED @COMPONENT (NOT @SERVICE)

UserServices

AdminServices ⚠️ bug in update() — ID not set before save

ProductServices

OrderServices

Logic.countTotal() (utility, not a service bean)

↓
Spring Data CrudRepository (findAll() returns Iterable — cast to List)

🗄️ REPOSITORIES — EXTEND CRUDREPOSITORY

UserRepository

AdminRepository

ProductRepository

OrderRepository

↓
JPA / Hibernate (H2 in dev, PostgreSQL in prod)

💾 DATABASE — H2 (IN-MEMORY, DEV) / POSTGRESQL (DOCKER / PROD)

customer_info — User

admin — Admin

product_table — Product

order_info — Orders ⚠️ field: totalAmmout (double-m typo)

↕
HttpSession (per-request state)

💡 HTTP SESSION — STORED IN JVM MEMORY

"loggedInAdmin" → String (email) "loggedInUser" → User entity (full JPA object)

5. CI/CD & Deployment

AWS CodeBuild (buildspec.yml)

The project includes a `buildspec.yml` for AWS CodeBuild. There is no GitHub Actions workflow present.

PHASE	COMMANDS
install	Checks Maven and Java versions (<code>mvn -v</code> , <code>java -version</code>)
build	<code>mvn clean package -DskipTests</code> — produces the JAR
post_build	Copies JAR to <code>output/</code> directory

Artifacts produced: `output/*.jar` , `appspec.yml` , `scripts/**/*` (for CodeDeploy).

Docker

A multi-stage `Dockerfile` is included:

- **Stage 1 (build)** — `eclipse-temurin:21-jdk-alpine` — resolves dependencies offline, builds the JAR with `./mvnw clean package -DskipTests` .
- **Stage 2 (run)** — `eclipse-temurin:21-jre-alpine` — minimal JRE image. Creates a non-root user (`spring:spring`) for security. Copies JAR and `entrypoint.sh` .

Docker Compose

Starts two services: `app` (the Spring Boot application) and `db` (PostgreSQL). The app waits for the database to be healthy before starting.

```
docker compose up -d
docker compose logs -f app
```

Environment Variables

VARIABLE	DEFAULT VALUE	PURPOSE
----------	---------------	---------

<code>SPRING_DATASOURCE_URL</code>	<code>jdbc:postgresql://db:5432/foodfiesta</code>	Database connection URL (overrides application.properties in Docker)
<code>SPRING_DATASOURCE_USERNAME</code>	<code>postgres</code>	Database username
<code>SPRING_DATASOURCE_PASSWORD</code>	<code>password</code>	Database password — change in production
<code>POSTGRES_DB</code>	<code>foodfiesta</code>	PostgreSQL database name (docker-compose db service)
<code>POSTGRES_USER</code>	<code>postgres</code>	PostgreSQL user (docker-compose db service)
<code>POSTGRES_PASSWORD</code>	<code>password</code>	PostgreSQL password (docker-compose db service)

Production warning: `SPRING_DATASOURCE_PASSWORD` and `POSTGRES_PASSWORD` default to `password`. Always override these in production via secrets manager or environment injection — never leave defaults.

Build Commands Reference

```
# Build Docker image
docker build -t food-fiesta .

# Run with Docker (H2 config overridden by ENV in Dockerfile)
docker run -p 8080:8080 --name food-fiesta-app food-fiesta

# Run app + PostgreSQL
docker compose up -d
```

6. Project Health & Technical Debt Tracking

A **technical debt audit** has been automatically run as part of this onboarding. The full report is available in the `docs/audit/` folder and provides a comprehensive view of the project's code quality, security gaps, architecture issues, and a prioritized refactoring backlog.

Re-running the Audit

To regenerate the full audit report at any time (with automatic delta vs. baseline):

Open a **new Bob conversation** on this project and type: **"lance un audit de dette technique"**

Temporal Tracking

The file `docs/audit/dette_technique_baseline.json` stores the reference snapshot. Re-running the audit compares against it and shows what has improved or regressed since the last run.

Audit Deliverables

FILE	DESCRIPTION
<code>docs/audit/dette_technique.html</code>	Interactive HTML report — KPI dashboard + prioritized backlog
<code>docs/audit/dette_technique.pdf</code>	PDF version with debt visualization chart
<code>docs/audit/dette_technique_baseline.json</code>	Temporal tracking baseline (JSON snapshot)

7. Known Gotchas & Contribution Guide

Known Gotchas

1. Intentional typo — `totalAmmout` (double-m): This field in the `Orders` entity and its getter/setter is intentionally misspelled. It maps to the `totalAmmout` DB column. *Never "fix" this typo* — it will break JPA column mapping and all existing data.

2. Services annotated `@Component` not `@Service`: `UserServices`, `AdminServices`, `ProductServices`, and `OrderServices` all use `@Component`. This works but goes against Spring convention. Follow the existing pattern for new services.

3. Plain-text passwords: All passwords (admin and user) are stored and compared in plain text. `BCryptPasswordEncoder` is declared as a Spring bean in `SecurityConfig` but is not injected anywhere. Do not assume passwords are hashed.

4. Mutation routes use `@GetMapping` : Routes like `/updatingAdmin/{id}` , `/updatingProduct/{productId}` , `/updatingUser/{id}` , and `/deleteAdmin/{id}` are all mapped to HTTP GET. This is the established pattern — do not change to PUT/DELETE/POST without updating the corresponding Thymeleaf templates and Spring Security config.

5. `findAll()` returns `Iterable` , not `List` : All repositories extend `CrudRepository` . `findAll()` returns `Iterable<T>` . Always cast to `List<T>` — e.g. `(List<User>) userRepository.findAll()` .

6. `Optional.get()` without guard: `AdminServices.getAdmin()` , `UserServices.getUser()` , and `ProductServices.getProduct()` call `optional.get()` without checking `isPresent()` first. Passing a non-existent ID throws `NoSuchElementException` . Be careful with any code path that takes user-supplied IDs.

7. Bug in `AdminServices.update()` : The method loops over all admins, finds the one matching the given ID, but then calls `save(admin)` without first calling `admin.setAdminId(id)` . The admin object passed in from the form has no ID set, so JPA inserts a new row rather than updating. This is a known bug — do not rely on admin update working correctly until it is fixed.

8. Session key types: `"loggedInAdmin"` stores a `String` (the email). `"loggedInUser"` stores a full `User` JPA entity. Always cast accordingly: `(User) session.getAttribute("loggedInUser")` and `(String) session.getAttribute("loggedInAdmin")` .

9. Use `Logic.countTotal()` for order total: Always use `Logic.countTotal(price, quantity)` from `com.example.demo.count` for order total computation. Do not inline `price * quantity` .

10. Google OAuth2 with placeholder credentials: The `application.properties` has `YOUR_CLIENT_ID` / `YOUR_CLIENT_SECRET` as placeholders. Leave them as-is for local H2 development — Google login simply won't work but the rest of the app is unaffected.

11. `OrderController` is an empty class: The file exists but contains no methods. All order logic (search, place order, dashboard) lives in `AdminController` .

Contribution Guide

TASK	COMMAND / ACTION
Run the app locally	<code>.\mvnw.cmd spring-boot:run</code>

Run all tests	<code>.\mvnw.cmd test</code>
Run a single test class	<code>.\mvnw.cmd test -Dtest=FoodFrenzyApplicationTests</code>
Package (skip tests)	<code>.\mvnw.cmd clean package -DskipTests</code>

- **All existing tests must pass** before pushing — run `.\mvnw.cmd test` and verify no failures.
- **Branching:** No CI pipeline detected on GitHub Actions. Infer from project structure: direct pushes to `main` are used. Consider creating feature branches and using PRs for larger changes.
- **New service method:** If you add a method to any `*Services.java`, add a matching test in the corresponding `*ServicesTest.java` file under `src/test/java/com/example/demo/services/`.
- **New controller endpoint:** Add `@Operation(summary=..., description=...)` and ensure the class has `@Tag(name=..., description=...)` — Swagger picks up everything in `com.example.demo` automatically.
- **New entity:** Add `@Table(name = "explicit_table_name")` — do not rely on Hibernate's default naming strategy.
- **Swagger / OpenAPI:** Accessible at `http://localhost:8080/swagger-ui/index.html` after startup — use it to test your endpoints quickly.

Reminder: "If you add a new service method, add a matching test in the corresponding `*Test.java` file."